

Process: A running program

Program:

- ↳ On the disk
- ↳ Doesn't do anything on it's own
- ↳ Contains bunch of instructions

The Crux of the Problem:

How to provide the illusion of many CPUs?

↳ The OS creates it by virtualizing the CPU. By running one process, then stopping it & running another.

↳ This basic technique known as "time sharing" allows the users to run as many concurrent processes as they would like.

Time sharing:

- ↳ One of the basic techniques used by OS to share resources.
- ↳ Allows a resource to be used for a little while by one entity and a little while by another.

Space sharing:

- ↳ A natural counterpart to time-sharing
- ↳ A resource is divided (in space) among those who wish to use it.

Example, disks, A block assigned to a file isn't used by other till deleted

↳ To implement virtualization,
↳ The OS will need some low level machinery called "mechanisms"

- ↳ These are low-level methods or protocols that implement a needed piece of func.
- ↳ Answers the "How" question.

Example,

How does the operating system perform context switching?

↳ We also need some high-level intelligence:

↳ They are in the form of "policies"

↳ Policies are algorithms for making some kind of decision within the OS.

↳ Answers the "What" Question

Example,

What process should OS run right now?

The Abstraction: A Process

↳ Process is simply a running program.

↳ We can summarize a process by taking an inventory of the different pieces of the system it accesses or affects during the course of its execution.

↳ Machine state of process:

↳ What a program can read/update when it is running?

↳ At any given time, what parts of the machine are important to the execution of a program?

1. Memory:

↳ Instructions, the data read by the running program & writes sit in memory.

↳ Thus, the memory that the process can address (called its address space) is part of the process.

2. Registers

↳ Stack pointer

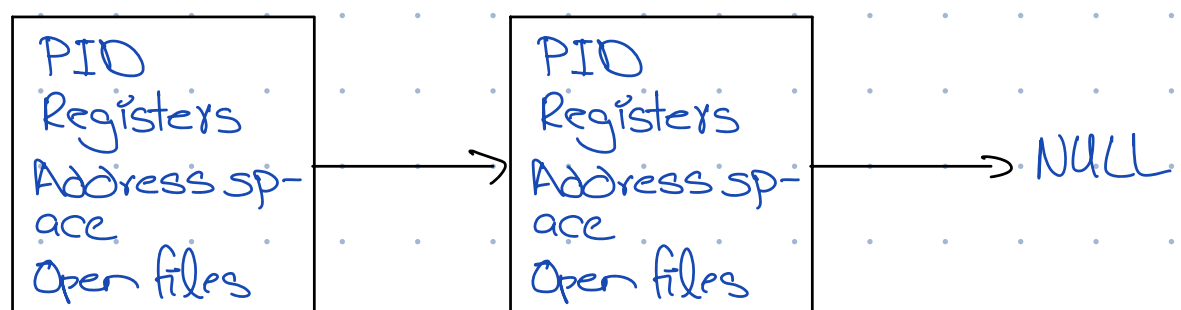
↳ Program counter

↳ Frame pointer

3. I/O information

↳ Programs often access persistent storage devices

↳ We track a list of files the process currently has open.



↳ This information is called context of a process

4. Unique Process Identifier (PID)

Process API:

- APIs provide endpoints, which are simply functions.
- Below are available on any modern system.

1. Create:

- An OS must include some method to create new processes.

2. Destroy:

- An OS must include some method to destroy processes.
- Many processes will run & exit when completed.
- When they don't, we may kill them.

3. Wait:

- Sometimes, it is useful to wait a process to stop running.

4. Miscellaneous control:

- There must be a way to:
 - Suspend
 - Resumea process.

5. Status:

- Provide a way to access info like how long it has run or what state it is in.

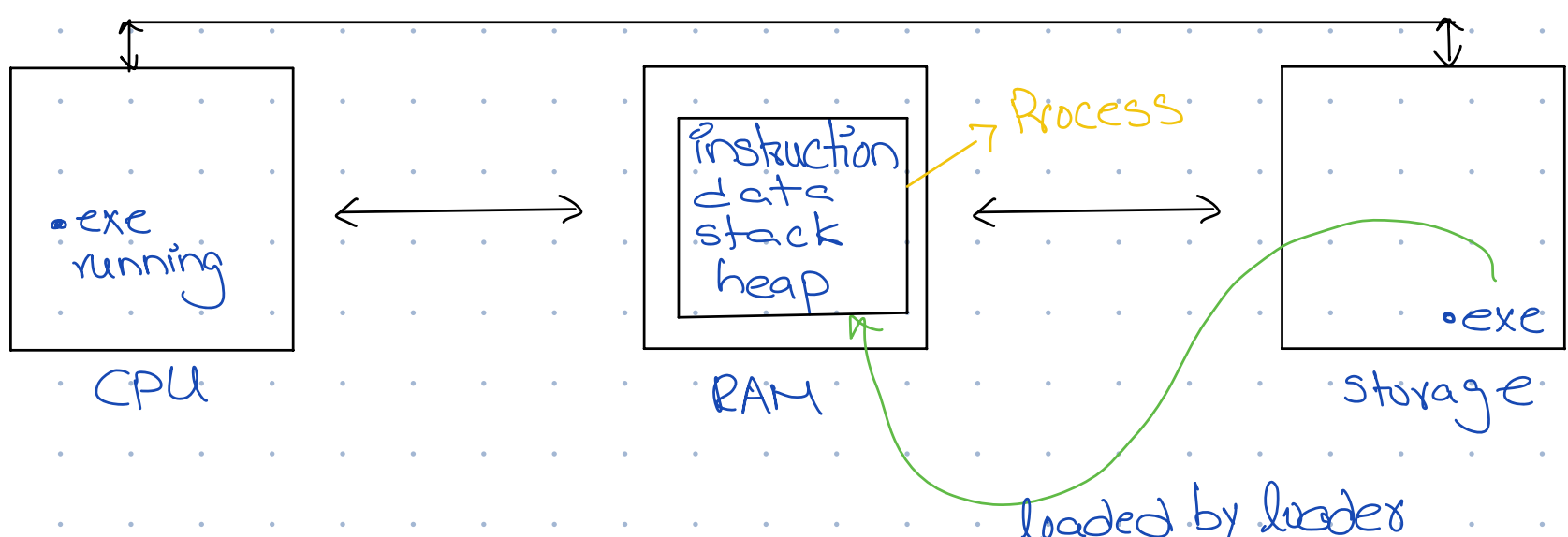
Process Creation:

"How are programs transformed into processes?"

0. Allocate memory

1. Loads the program & static data into memory

- Programs reside in storage in some executable format.



- ↳ In early OS, the loading process is done all at once before running the program.
- ↳ In modern OSes, It only loads parts of code & data it needs during execution.

2. Allocate memory for the program's stack

↳ Refer to How variables are stored?

How variables are stored?

```
#define X 100
```

```
char *s = "Hello";
```

```
int f()
{
```

```
    char a = 'A';
```

```
    char b = a + 1;
```

```
    g(a)
```

```
}
```

```
int g(char y)
{
```

```
    int z = 0;
```

```
    if (y == 'A')
```

```
        z = 1;
```

```
    else
```

```
        z = 0;
```

```
    return z;
```

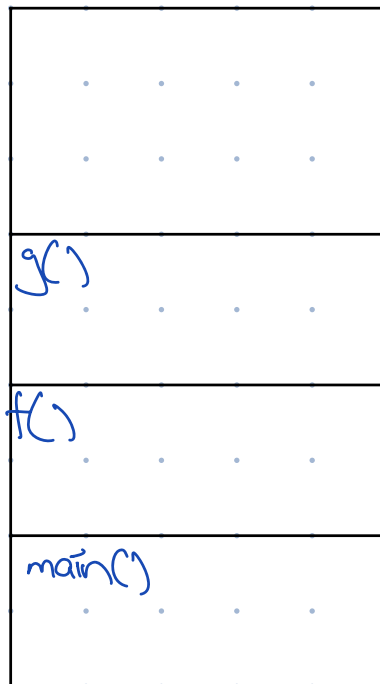
```
}
```

→ The compiler inserts a code above this function call that gives the total size of stack needed.

↳ size of arguments + local variables + return address

↳ Only space is allocated.

↳ Global variables (X) are not stored like this.



} size might be 60

} This might have 50

⋮

3. Create some initial memory for the program's heap.

↳ This space is used for explicitly requested dynamically allocated data.

↳ This heap is small at first.

↳ As program runs & asks for more, OS increases the size.

4. Initialize files related to I/O:

↳ For unix systems, each process by default has three open file descriptions, for standard input, output and error.

5. Initialize registers (context)

Process states:

↳ A process can be in one of three states:

1. Running

↳ The process is on a processor.

↳ This means it is being executed

2. Ready:

↳ In this, the process is ready to run but the OS has chosen not to for some reason.

3. Blocked:

↳ In this, a process has performed some kind of operation that makes it not ready until some other event takes place.

Example,

When a process initiates an I/O request to a disk, it becomes blocked & thus some other process uses the processor.

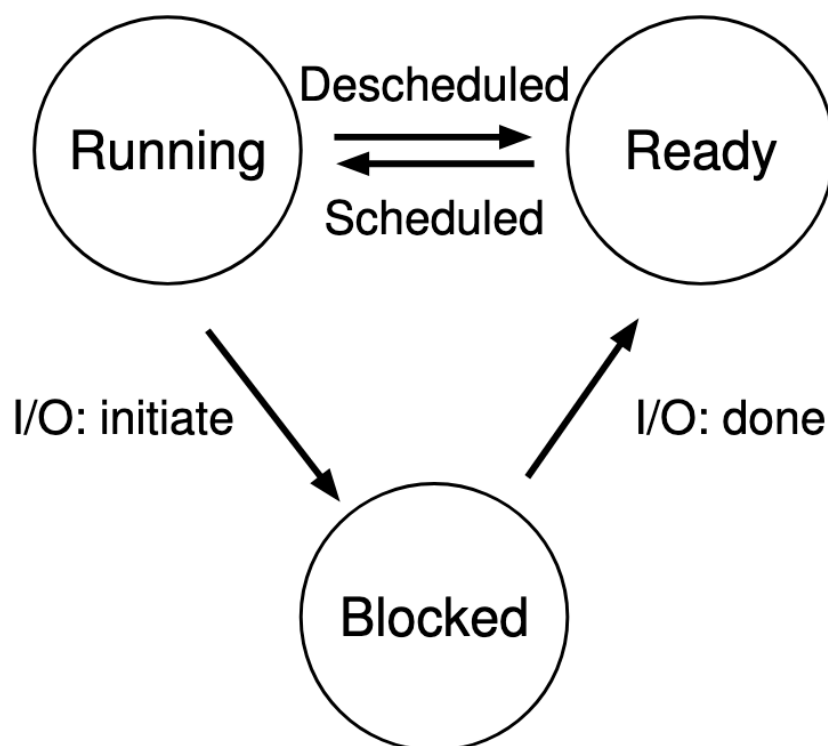


Figure 4.2: Process: State Transitions

Data Structures:

- ↳ The OS is a program, uses Data structures to keep track of some information.
- ↳ To track state of each process, the OS keeps some kind of process list for all processes.
- ↳ It also keeps track of running & blocked processes
- ↳ In particular,
 - ↳ The register context will hold, for a stopped process, the contents of its registers.
 - ↳ When a process is stopped, its register will be saved to memory.
- ↳ There are some other states a process can be in:

1. Initial state:

- ↳ When a process is created

2. Final state:

- ↳ This state is useful as it allows other processes (parent) to examine the return code. (zero if successfully else non-zero).
- ↳ When finished parent makes a wait() call to wait for completion of a child process.

3. Zombie / Dead state

- ↳ Where the process has exited but not yet cleaned up.

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
    int eip;
    int esp;
    int ebx;
    int ecx;
    int edx;
    int esi;
    int edi;
    int ebp;
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };

// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem;                // Start of process memory
    uint sz;                  // Size of process memory
    char *kstack;             // Bottom of kernel stack
                              // for this process
    enum proc_state state;    // Process state
    int pid;                  // Process ID
    struct proc *parent;      // Parent process
    void *chan;               // If non-zero, sleeping on chan
    int killed;               // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;         // Current directory
    struct context context;    // Switch here to run process
    struct trapframe *tf;     // Trap frame for the
                              // current interrupt
};
```

Figure 4.3: The xv6 Proc Structure

Process Control Block (PCB):

↳ Individual data structures that store information about a process